

Practical No. 10

Name : Krushna Santosh Pawar

Class : SY-B

Branch – AI & DS

Roll no : B-27

Aim - Version Control using Git & GitHub Questions :

1. What is the difference between Git and GitHub? Ans

: Git vs GitHub

- Git is a Version Control System (VCS)
- Created by Linus Torvalds in 2005
- Works completely offline on your local machine
- Tracks every change made to your code
- Stores full project history locally
- Free and open-source
- Used via command line (terminal)
- Alternatives: Mercurial, SVN
- GitHub is a cloud-based hosting platform for Git repos
- Founded in 2008, bought by Microsoft in 2018
- Requires internet connection
- Adds collaboration features: Pull Requests, Issues, Actions
- Stores your code remotely on servers
- Has free and paid plans
- Alternatives: GitLab, Bitbucket

2. Explain the difference between:

- **git add**

- **git commit**

Ans : git add:

- Moves files from Working Directory → Staging Area
- Tells Git which changes to include in next commit
- Does NOT save permanently
- Can stage one file, multiple files, or all files
- Reversible using git restore –staged

Commands:

Git add file.txt → one file

Git add . → all files

Git add *.js → all JS files

Git add -p → select parts interactively Git commit:

- Moves files from Staging Area → Local Repository
- Creates a permanent snapshot in history
- Each commit gets a unique SHA-1 hash ID
- Requires a commit message • Records author, timestamp, and changes

Commands:

Git commit -m “message” → basic commit Git

commit –amend → edit last commit Git commit am

“message” → add + commit together

3.What is the staging area in Git?

Ans : • Also called the Index

- It is a middle layer between working directory and repository
- Physically stored in .git/index file
- Lets you selectively choose what to commit
- You can modify 10 files but commit only 2
- Helps create clean, logical commits • You can review staged changes before saving

- Commands:

Git status → see staged/unstaged files Git

diff --staged → see staged changes Git

restore --staged file → unstage a file -

4 States of a file:

- Untracked → new file Git doesn't know about
- Modified → tracked file with new changes
- Staged → ready to be committed
- Committed → permanently saved in history

4. What happens if you modify a file after committing it?

Ans : • The old committed version stays safe in history

- Modified file appears as “modified” in working directory
- Git detects change by comparing file hash with last commit
- Git status shows it under “Changes not staged for commit”
- You must add and commit again to save new version
- Each commit gets a new unique hash
- Old commit is still accessible anytime
- You can go back using git checkout <old-hash>
- Nothing in Git is truly lost after committing
- To fix last commit quickly: git commit --amend Process after modifying:

Modify file → git add → git commit → new snapshot created

5. Difference between:

- git push

- git pull

Ans : git push:

- Uploads local commits → remote repository (GitHub)

- Shares your work with teammates
- Only pushes committed changes (not just staged)
- May be rejected if remote has newer commits Commands:
 - Git push origin main → push to main branch
 - Git push origin branch-name → push specific branch
 - Git push -u origin main → set default upstream
 - Git push -force → force overwrite (dangerous)

Git
pull:

- Downloads remote changes → local machine
- Keeps your local repo up to date
- Combines: git fetch + git merge
- Can cause merge conflicts if same lines were edited Commands:
 - Git pull origin main → pull from main
 - Git pull -rebase → pull and rebase instead of merge

6 .What is Version Control? Differentiate between Centralized v/s Distributed VCS.

Ans : Version Control:

- System that records changes to files over time
- Tracks who, what, when, and why something changed
- Allows reverting to any previous version
- Enables parallel work without overwriting each other • Essential for team software development Centralized VCS (CVCS):
- One central server holds the entire repository
- Developers check out only one version (not full history)
- All operations need server/internet connection
- Examples: SVN, CVS, Perforce - Advantages:
- Simple to understand

- Admin has full control
- Good for binary/design files Distributed VCS (DVCS):
- Every developer has full copy of entire repo including history
- Most operations happen locally (fast) • Remote server used only for syncing/sharing
- Examples: Git, Mercurial - Advantages:
- Fully offline capability
- Every clone = complete backup
- Very fast operations
- Easy branching and merging • No single point of failure **Theroy :**

```

MINGW64/c/Users/Suraj
Suraj@LAPTOP-UVIMLLP MINGW64 ~
$ git --version
git version 2.54.0.windows.1

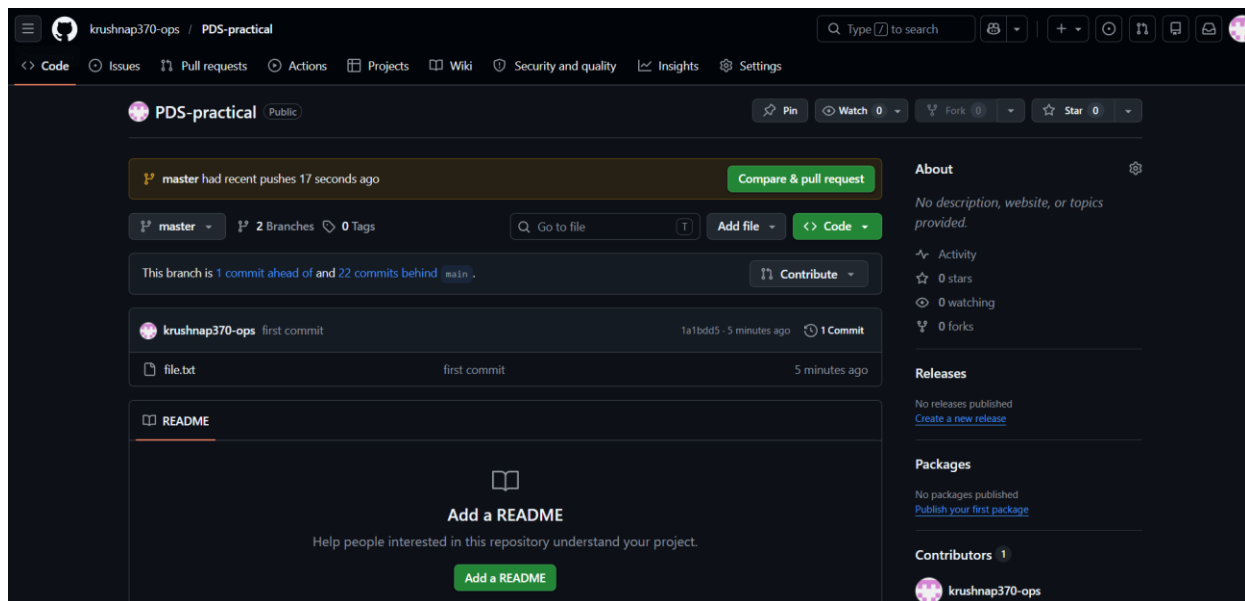
Suraj@LAPTOP-UVIMLLP MINGW64 ~
$ git config --global user.name"krushna"

Suraj@LAPTOP-UVIMLLP MINGW64 ~
$ git config --global user.email"krushnap370@gmail.com"

Suraj@LAPTOP-UVIMLLP MINGW64 ~
$ git config --list

diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master

```



```
Suraj@LAPTOP-UVIMLLP MINGW64 ~
$ mkdir myproject

Suraj@LAPTOP-UVIMLLP MINGW64 ~
$ cd myproject

Suraj@LAPTOP-UVIMLLP MINGW64 ~/myproject
$ git init
Initialized empty Git repository in C:/Users/Suraj/myproject/.git/

Suraj@LAPTOP-UVIMLLP MINGW64 ~/myproject (master)
$ touch file.txt

Suraj@LAPTOP-UVIMLLP MINGW64 ~/myproject (master)
$ git add file.txt

Suraj@LAPTOP-UVIMLLP MINGW64 ~/myproject (master)
$ git add .

Suraj@LAPTOP-UVIMLLP MINGW64 ~/myproject (master)
$ git commit -m"first commit"
[master (root-commit) 1a1bdd5] first commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 file.txt

Suraj@LAPTOP-UVIMLLP MINGW64 ~/myproject (master)
$ git remote add origin https://github.com/krushnap370-ops/PDS-practical.git

Suraj@LAPTOP-UVIMLLP MINGW64 ~/myproject (master)
$ git push -u origin master
fatal: unable to access 'https://github.com/krushnap370-ops/PDS-practical.git/': Could not resolve host: github.com

Suraj@LAPTOP-UVIMLLP MINGW64 ~/myproject (master)
$ git push -u origin master
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 208 bytes | 208.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:   https://github.com/krushnap370-ops/PDS-practical/pull/new/master
remote:
to https://github.com/krushnap370-ops/PDS-practical.git
* [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

krushnap370-ops / PDS-practical

Search

Type / to search

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security and quality

Insights

Settings

PDS-practical

Public

Pin

Watch 0

Fork 0

Star 0

master had recent pushes 33 seconds ago

Compare & pull request

main

2 Branches

0 Tags

Go to file

Add file

Code

krushnap370-ops

Delete Untitled2.ipynb

bb3b48a · 23 minutes ago

22 Commits

README.md	Initial commit	yesterday
pr1.ipynb	Created using Colab	yesterday
pr2.ipynb	Created using Colab	yesterday
pr3.ipynb	Created using Colab	yesterday
pr4.ipynb	Created using Colab	yesterday
pr5.ipynb	Created using Colab	yesterday
pr6.ipynb	Created using Colab	yesterday
pr7.ipynb	Created using Colab	yesterday
pr9_ipnyb.ipynb	Created using Colab	yesterday

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 1